

LRU Page Replacement Algorithm

Write a C program to implement the LRU Page Replacement Algorithm. The program should accept a reference string and the number of frames. For each page request, identify whether it results in a Hit or a Fault. If a Fault occurs and the frames are full, replace the page that was accessed least recently. Display the frame status at each step and calculate the total number of Page Faults and Hits

Input Format

1. The first line of input contains an integer representing the number of pages in the reference string.
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

Output Format

The output displays the total number of Page Faults and the total number of Page Hits after processing the entire reference string using the LRU page replacement algorithm.

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, frames, i, j, k, pos;
5
6     printf("Enter number of pages in reference string: \n");
7     scanf("%d", &n);
8
9     int ref[n];
10
11    printf("Enter the reference string: \n");
12    for(i = 0; i < n; i++) {
13        scanf("%d", &ref[i]);
14    }
15
16    printf("Enter number of frames: \n");
17    scanf("%d", &frames);
18
19    int frame[frames];
20    time[frames];
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

LRU Page Replacement Algorithm

Write a C program to implement the LRU Page Replacement Algorithm. The program should accept a reference string and the number of frames. For each page request, identify whether it results in a Hit or a Fault. If a Fault occurs and the frames are full, replace the page that was accessed least recently. Display the frame status at each step and calculate the total number of Page Faults and Hits

Input Format

1. The first line of input contains an integer representing the number of pages in the reference string. \
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

Output Format

The output displays the total number of Page Faults and the total number of Page Hits after processing the entire reference string using the LRU page replacement algorithm.

Completed

Select Language : C (GCC 9.2.0)

```
19  int frame[frames], time[frames];
20
21  for(i = 0; i < frames; i++) {
22      frame[i] = -1;
23      time[i] = 0;
24  }
25
26  int faults = 0, hits = 0, counter = 0;
27
28  for(i = 0; i < n; i++) {
29      int found = 0;
30
31      // check for hit
32      for(j = 0; j < frames; j++) {
33          if(frame[j] == ref[i]) {
34              hits++;
35              counter++;
36              time[j] = counter;
37              found = 1;
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

LRU Page Replacement Algorithm

Write a C program to implement the LRU Page Replacement Algorithm. The program should accept a reference string and the number of frames. For each page request, identify whether it results in a Hit or a Fault. If a Fault occurs and the frames are full, replace the page that was accessed least recently. Display the frame status at each step and calculate the total number of Page Faults and Hits

Input Format

1. The first line of input contains an integer representing the number of pages in the reference string. \
2. The second line contains the page reference string as a sequence of integers.
3. The third line contains an integer representing the number of available frames.

Output Format

The output displays the total number of Page Faults and the total number of Page Hits after processing the entire reference string using the LRU page replacement algorithm.

Select Language : C (GCC 9.2.0)

```
46 // Find least recently used
47 for(j = 1; j < frames; j++) {
48     if(time[j] < time[pos]) {
49         pos = j;
50     }
51 }
52
53 frame[pos] = ref[i];
54 counter++;
55 time[pos] = counter;
56 faults++;
57 }
58 }
59
60 printf("\nTotal Page Faults: %d\n", faults);
61 printf("Total Page Hits: %d\n", hits);
62
63 return 0;
64 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1